

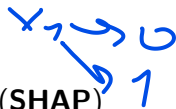
Why this one prediction?



Global importance (deck 2) says temp and yr matter *overall*. But the model predicts **this particular day** at **4311 rentals** (below the ~ 4500 average) — which features drove *that*?



Global tools can't answer per-prediction. Shapley Additive Explanations (SHAP) can: it explains a **single prediction**, and the same idea rolls up into global plots.



Still the bike-sharing random forest from decks 1–2.

Outline

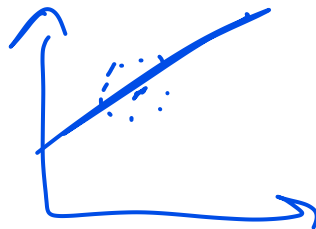
What SHAP is ✓

The SHAP plots (how to read them) ✓

SHAP in practice

LIME

Counterfactual explanations ✓



Hand-drawn blue scribble or signature.

What SHAP is

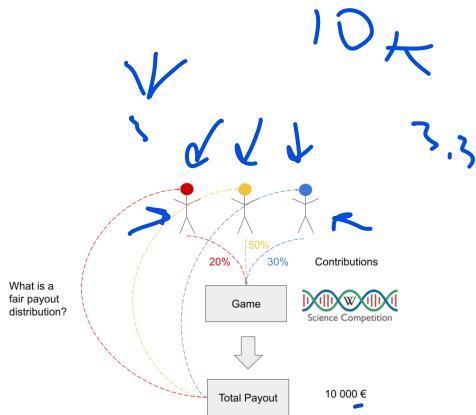
A fair split of the prediction among the features.

The idea: a fair payout

Treat the prediction as a **payout to split fairly** among the features — like a team bonus among players who contributed differently.

SHAP gives each feature its **fair share**: its contribution *averaged over all the orders* in which features could be added to the model.

Next: where this idea comes from, then we work one out *by hand* before meeting the formula.



Source: LMU IML (CC BY 4.0).

This is not an ML idea: it comes from game theory

The **Shapley value** was defined by **Lloyd Shapley in 1953**, in **cooperative game theory** — decades before anyone used it on a model.

The setup there:

- ▶ A set of **players** $P = \{1, \dots, p\}$; any subset $S \subseteq P$ can form a **coalition**.
- ▶ A **value function** $v(S)$ says what each coalition earns, with $v(\emptyset) = 0$. The whole team earns $v(P)$.
- ▶ Player j 's **marginal contribution** to a coalition:

$$\Delta(j, S) = v(S \cup \{j\}) - v(S)$$

- ▶ **The question:** how do we split $v(P)$ fairly, given players contribute differently and help each other?

The translation to ML:

players $\rightarrow$ features

coalition $S \rightarrow$ a **subset** of features

payout $v(P) \rightarrow \hat{f}(\mathbf{x}) - \mathbb{E}[\hat{f}]$

fair share $\phi_j \rightarrow$ **SHAP value**

Shapley shared the **2012 Nobel Memorial Prize in Economics** with Alvin Roth — though for *stable matching and market design*, not for this value.

By hand: three friends split the payout

Adi, Budi and Gogi run a stall together.
Working in different groups, they would earn
different amounts — the group's **payout**
 $v(S)$:

Together they earn **90**. What is each one's
fair share of that 90?

Names: garakusi.am

who works together S	$v(S)$
nobody	0
<u>Adi</u>	<u>10</u>
Budi	<u>20</u>
Gogi	30
<u>Adi + Budi</u>	<u>40</u>
Adi + Gogi	50
Budi + Gogi	<u>60</u>
<u>all three</u>	<u>90</u>

no

2

$$40 = 10 + 20$$

B C, C, B

Fair share = average marginal contribution

Adi's *extra* contribution depends on who is already there:

who's before <u>Adi</u>	Δ_{Adi}	#orders
nobody (Adi first)	$10 - 0 = 10$	2
Budi	$40 - 20 = 20$	1
Gogi	$50 - 30 = 20$	1
Budi & Gogi	$90 - 60 = 30$	2

Average over all **6 orders**:

$$\phi_{\text{Adi}} = \frac{2(10) + 20 + 20 + 2(30)}{6} = 20$$

1 1 1 3!

1 1 3
LOCO

Same for each friend:

$$\phi_{\text{Adi}}=20, \quad \phi_{\text{Budi}}=30, \quad \phi_{\text{Gogi}}=40$$

They **sum to 90** — the whole payout is split, nothing lost or invented (*efficiency*).

In a model: the players are **features**, the payout is $\hat{f}(\mathbf{x}) - \mathbb{E}[\hat{f}]$, and each fair share is that feature's **SHAP value** ϕ_j .

2 3
Tree Sh
Keras

Now the same thing, formally

You just computed a Shapley value by hand. The next two slides are that *exact* procedure written in general notation — nothing new, just symbols for what we already did.

Watch for: the **value function** $v(S)$ — that was our payout table — and the **marginal contribution** $v(S \cup \{j\}) - v(S)$, which was Adi's “extra” column.

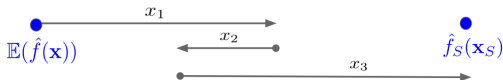
The scary-looking $\frac{|S|!(p-|S|-1)!}{p!}$ is just the **#orders** column from our table, turned into a fraction — how often that coalition comes up among all $p!$ orders.

FROM GAME THEORY TO MACHINE LEARNING

- Game: Make prediction $\hat{f}(x_1, x_2, \dots, x_p)$ for a single observation $\mathbf{x}$
- Players: Features $x_j, j \in \{1, \dots, p\}$ which cooperate to produce a prediction
 - $\rightsquigarrow$ How can we make a prediction with a subset of features without changing the model?
 - $\rightsquigarrow$ PD function: $\hat{f}_S(\mathbf{x}_S) := \int_{\mathcal{X}_{-S}} \hat{f}(\mathbf{x}_S, X_{-S}) d\mathbb{P}_{X_{-S}}$ ("removing" by marginalizing over $-S$)
- Value function / payout of coalition $S \subseteq P$ for observation $\mathbf{x}$:

$$v(S) = \hat{f}_S(\mathbf{x}_S) - \mathbb{E}_{\mathbf{x}}(\hat{f}(\mathbf{x})), \text{ where } \hat{f}_S : \mathcal{X}_S \mapsto \mathcal{Y}$$

$\rightsquigarrow$ subtraction of $\mathbb{E}_{\mathbf{x}}(\hat{f}(\mathbf{x}))$ ensures that v is a value function with $v(\emptyset) = 0$



- Marginal contribution: $v(S \cup \{j\}) - v(S) = \hat{f}_{S \cup \{j\}}(\mathbf{x}_{S \cup \{j\}}) - \hat{f}_S(\mathbf{x}_S)$
 - $\rightsquigarrow \mathbb{E}_{\mathbf{x}}(\hat{f}(\mathbf{x}))$ cancels out due to the subtraction of value functions

SHAPLEY VALUE - DEFINITION

► Shapley (1953)

► Strumbelj et al. (2014)

Shapley value ϕ_j of feature j for observation $\mathbf{x}$ via **order definition**:

$$\phi_j(\mathbf{x}) = \frac{1}{|P|!} \sum_{\tau \in \Pi} \underbrace{\hat{f}_{S_j^\tau \cup \{j\}}(\mathbf{x}_{S_j^\tau \cup \{j\}}) - \hat{f}_{S_j^\tau}(\mathbf{x}_{S_j^\tau})}_{\text{marginal contribution of feature } j}$$

- Interpretation: Feature x_j contributed ϕ_j to difference between $\hat{f}(\mathbf{x})$ and average prediction
 $\rightsquigarrow$ Note: Marginal contributions and Shapley values can be negative
- For exact computation of $\phi_j(\mathbf{x})$, the PD function $\hat{f}_S(\mathbf{x}_S) = \frac{1}{n} \sum_{i=1}^n \hat{f}(\mathbf{x}_S, \mathbf{x}_{-S}^{(i)})$ for any set of features S can be used which yields

$$\phi_j(\mathbf{x}) = \frac{1}{|P|! \cdot n} \sum_{\tau \in \Pi} \sum_{i=1}^n \hat{f}(\mathbf{x}_{S_j^\tau \cup \{j\}}, \mathbf{x}_{-\{S_j^\tau \cup \{j\}\}}^{(i)}) - \hat{f}(\mathbf{x}_{S_j^\tau}, \mathbf{x}_{-S_j^\tau}^{(i)})$$

$\rightsquigarrow$ Note: $\hat{f}_S$ marginalizes over all other features $-S$ using all observations $i = 1, \dots, n$

One prediction = base value + the SHAP values

Every SHAP explanation is one simple sum:

The diagram shows the equation $\hat{f}(\mathbf{x}) = \mathbb{E}[\hat{f}(X)] + \sum_j \phi_j$ with handwritten blue annotations. A circle around $\hat{f}(\mathbf{x})$ is labeled "this prediction". A circle around $\mathbb{E}[\hat{f}(X)]$ is labeled "base value (avg. prediction)". A circle around $\sum_j \phi_j$ is labeled with the text "SHAP values" written vertically. Blue arrows point from the text "Every SHAP explanation is one simple sum:" to each of the three terms in the equation. A curved blue arrow also points from the first term to the second.

$$\underbrace{\hat{f}(\mathbf{x})}_{\text{this prediction}} = \underbrace{\mathbb{E}[\hat{f}(X)]}_{\text{base value (avg. prediction)}} + \underbrace{\sum_j \phi_j}_{\text{SHAP values}}$$

where ϕ_j is feature j 's SHAP value.

- ▶ $\phi_j > 0$ pushes this prediction **up**; $\phi_j < 0$ pushes it **down**.
- ▶ **Every SHAP plot visualizes this sum** — for one row, or aggregated over many.

One catch: what does “without a feature” even mean?

The coalition story needs $v(S)$: what the model predicts *using only* the features in S . But a **fitted model has a fixed input shape** — you cannot hand a trained forest 6 of its 11 columns.

So “absent” is **faked**: fill the missing features from a **background dataset** and average the predictions,

$$v(S) = \mathbb{E}[\hat{f}(\mathbf{x}_S, X_{\bar{S}})].$$

Replacing a feature *independently of the others* invents rows that never existed (30 °C and snow) — **deck 2’s extrapolation trap**, back again. It is the same reason correlated features muddy SHAP.

“Absent” is whatever your data type allows:

table resample the column

text mask the word out

image grey the patch out

That is the whole reason SHAP ports to text and images at all — we come back to this.

- ▶ No features at all $\Rightarrow v(\emptyset) = \mathbb{E}[\hat{f}]$: exactly the **base value** from the previous slide.
- ▶ So every ϕ_j is **relative to that background** — the X in `shap.Explainer(model, X)`.



0.7
0.3



Exact SHAP explodes — TreeSHAP & KernelSHAP to the rescue

3 features $\rightarrow 3! = 6$ orders ($2^3 = 8$ coalitions): easy by hand. But it **explodes** with the number of features:

features	orders $n!$
3	6
10	3.6 million
20	2.4×10^{18}

You **cannot** enumerate them all $\Rightarrow$ we **approximate**.

KernelSHAP: *sample* a handful of coalitions and fit a **weighted linear model** to recover the ϕ_j . Works on **any** model — but approximate and slow.

TreeSHAP: exploits the **tree structure** to get **exact** SHAP in *polynomial* time — fast + exact for trees/boosting $\Rightarrow$ the **default**.

The SHAP plots — how to read them

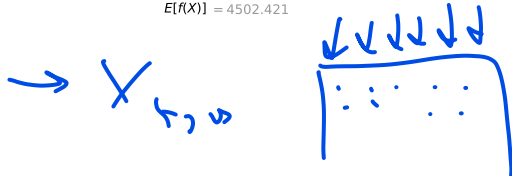
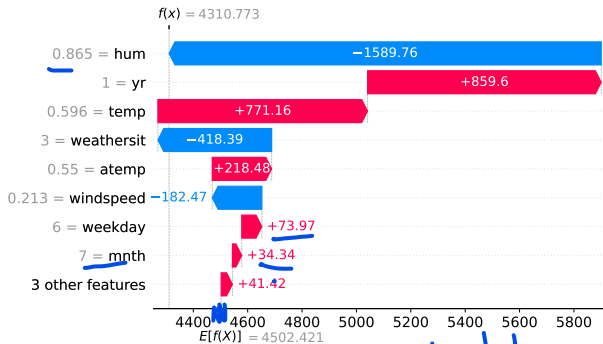
One local plot, three global ones. Same SHAP values, different views.

Waterfall: one prediction, fully explained

Start at the **base value** (~ 4502 rentals) and add each feature's SHAP value until you reach **this day's prediction** (4311).

Read it: yr (+860) and temp (+771) push it **up**, but a very humid day (hum -1590) and poor weathersit (-418) push it **down** — landing just below average.

Careful: $\phi_{\text{temp}} = +771$ does **not** mean “drop temp and the prediction falls by 771”. It is temp's *average* share over all the orders features could enter in — a **fair split**, not a what-if.

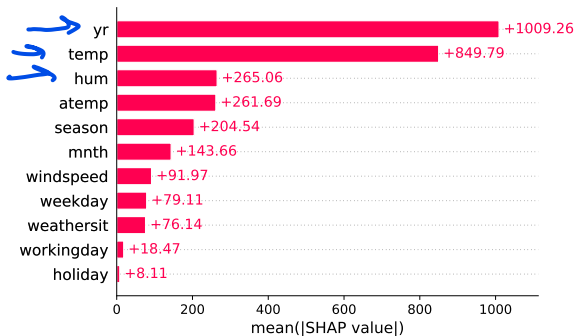


Bar: global importance (mean |SHAP|)

Average the **magnitude** of each feature's SHAP over all rows:

$$\text{importance}_j = \frac{1}{n} \sum_i |\phi_j^{(i)}|$$

- One bar per feature — **reads like the PFI bar** from deck 2.
- yr, temp dominate — roughly agreeing with PFI.



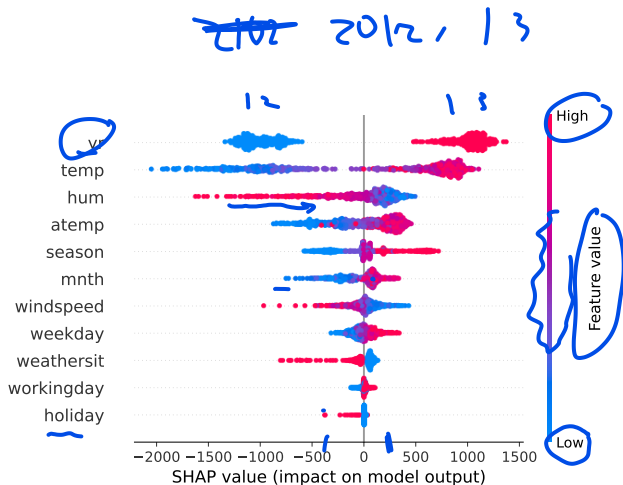
Beeswarm: the signature SHAP plot

One dot per row, per feature.

x = SHAP value; **colour** = feature value.

Read it:

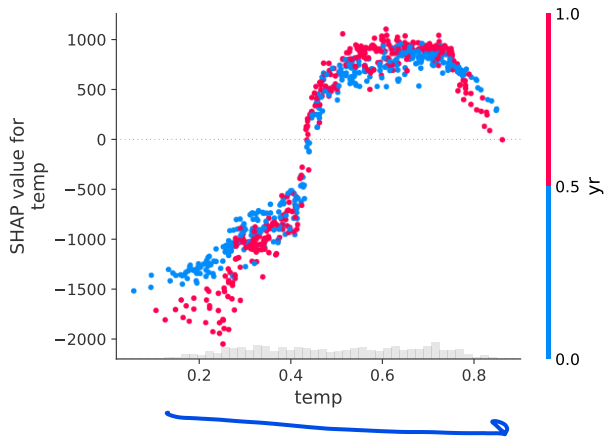
- **Spread** = how much the feature matters.
- **Colour pattern** = direction: high temp (red) → positive SHAP (more rentals); low (blue) → negative.



Dependence: the effect shape (& interactions)

SHAP value of a feature **vs. its value** — one dot per row.

- Shows the **effect shape**: rentals **rise sharply** with temp, peak, then **dip at extreme heat** ($\approx$ a PDP).
- Colour by a second feature (here yr) reveals **interactions**.
- Per-instance — *like ICE did for PDP*, it shows the spread the average hides.



How SHAP ties the methods together

- ▶ **TreeSHAP** (previous slide) makes SHAP the **default** for trees/boosting.
- ▶ **SHAP unifies what we've seen:**
 - ▶ the bar $\approx$ PFI (global importance),
 - ▶ the dependence plot $\approx$ PDP (effect shape),
 - ▶ **plus** a local view (the waterfall) the others can't give — with a consistency guarantee (use a feature more $\Rightarrow$ its SHAP can't fall).

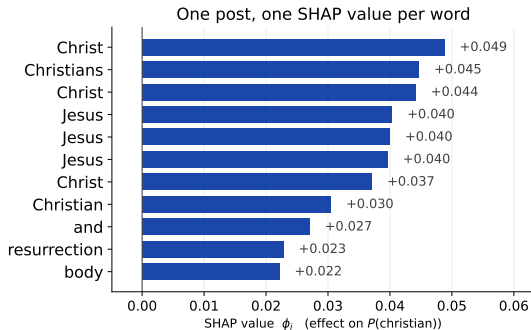
But like every method here, correlation muddies SHAP too — the temp/atemp credit split stays ambiguous. SHAP also explains the model's *score*: if that score isn't **calibrated** (L14), SHAP faithfully explains a *miscalibrated* number. Still **model behaviour, not causation** (deck 2's ice cream), and rich plots are **easy to over-read**.

SHAP on text: the same sum, over words

A post $\rightarrow$ TF-IDF $\rightarrow$ logistic regression: the “features” are now **words**, “absent” means **masked out of the string**, and **nothing else changes**.

$$\underbrace{0.41}_{\text{base}} + \underbrace{0.47}_{\sum_j \phi_j} = \underbrace{0.88}_{P(\text{christian})}$$

- One bar per **token position** — hence three Jesus bars.
- Hundreds of tokens $\Rightarrow 2^n$ coalitions: shap masks *groups* of neighbouring words (a **Partition** explainer).



$$\text{base value } 0.41 + \sum_j \phi_j = +0.47 = 0.88 = \hat{f}(x) \rightarrow \text{christian}$$

TF-IDF = *term frequency* $\times$ *inverse document frequency*: one column per vocabulary word, rare words weighted higher (properly in ch. 6). Logistic regression was deck 1's **glass box** — but nobody reads $\sim 16\,000$ coefficients. **An interpretable model class is not an interpretable model.**

SHAP in code

```
import shap
explainer = shap.TreeExplainer(model)           # fast & exact for trees/
    boosting
sv = explainer(X)                               # SHAP values for every
    row

shap.plots.waterfall(sv[i])                     # one prediction
shap.plots.bar(sv)                             # global: mean |SHAP|
shap.plots.beeswarm(sv)                        # global: dot per row,
    colour = value
shap.plots.scatter(sv[:, "temp"])               # dependence: effect
    shape

# text: same idea, features = words, "absent" = masked out of the
    string
ex = shap.Explainer(text_clf.predict_proba, shap.maskers.Text())
shap.plots.text(ex(["a review to explain"]))
```

shap.Explainer(model, X) auto-picks the right method (TreeSHAP here); with a Text masker it switches to the token version.

LIME: the lighter local cousin

A simple, model-agnostic local explanation — with sharper edges.



LIME: the locality assumption

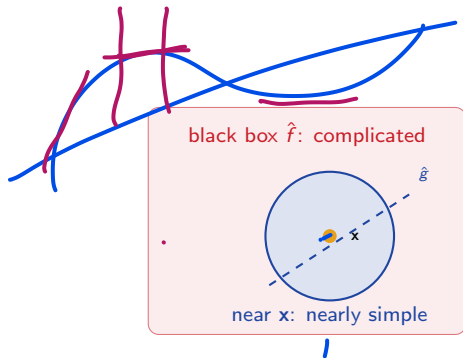
Local Interpretable Model-agnostic Explanations (LIME). Every word in that name is a design decision, and the next two slides unpack them.

The starting point: a black box $\hat{f}$ can be hopelessly complicated *globally* — but **in a small neighbourhood of one point x it is usually close to simple**.

So: don't explain $\hat{f}$. Explain a **local surrogate** $\hat{g}$ — an interpretable model (sparse linear, shallow tree) that mimics $\hat{f}$ *near x only*.

A surrogate is useful only if it is both:

1. **Interpretable** — you can read it (coefficients, structure).
2. **Locally faithful** — $\hat{g}(z) \approx \hat{f}(z)$ for points z *near x*



The goal: *minimum complexity, maximum local fidelity.*



The three ingredients you must choose

1. A **proximity measure** $\pi(\mathbf{z})$ — how much does the point $\mathbf{z}$ count as “near $\mathbf{x}$ ”? Usually an exponential kernel:

$$\pi(\mathbf{z}) = \exp(-d(\mathbf{x}, \mathbf{z})^2 / \sigma^2)$$

d = a distance (Euclidean, or **Gower** for mixed types); σ = the **kernel width**, which sets how local “local” is.

2. A **loss** $L(\hat{f}(\mathbf{z}), \hat{g}(\mathbf{z}))$ — usually squared error. Weighted by proximity, the **local fidelity** of $\hat{g}$ is

$$L(\hat{f}, \hat{g}, \pi) = \sum_{\mathbf{z} \in \mathcal{Z}} \pi(\mathbf{z}) (\hat{g}(\mathbf{z}) - \hat{f}(\mathbf{z}))^2$$

3. A **complexity measure** $J(\hat{g})$ — e.g. the number of non-zero coefficients of a linear surrogate, or the number of leaves of a tree.

LIME's optimization problem:

$$\hat{g} = \arg \min_{g \in \mathcal{G}} L(\hat{f}, g, \pi) + J(g)$$

Fit the black box closely nearby, while staying simple.

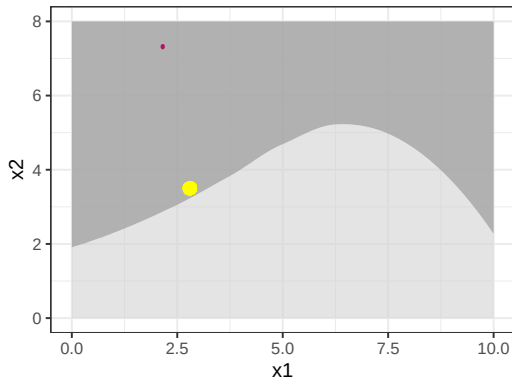
In practice nobody optimizes both: you **fix** the complexity first (“Lasso, keep k features”) and then just minimise the weighted loss.

Step 1: the point we want to explain

A classifier's decision surface (light vs dark grey), and one **yellow point x** whose prediction we want explained.

Globally that boundary is curved and messy — no linear model could describe all of it.

But look only at the little region *around the yellow point*: there, a straight line would do fine. That is the locality assumption, in a picture.



Source: LMU IML (CC BY 4.0).

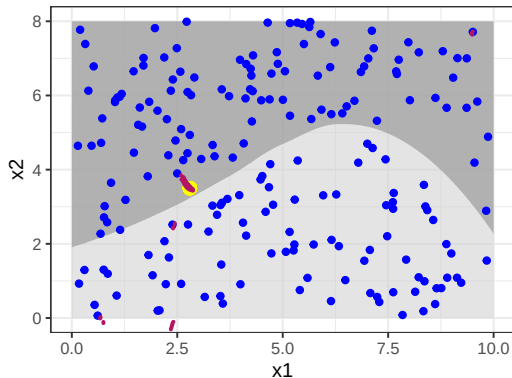
Step 2: sample new points, and label them with the black box

We need data to fit the surrogate on —
so we **make some**:

- Sample points $\mathbf{z}$ across the feature space (uniformly, on a grid, or by perturbing the training data).
- Ask the **black box** for its prediction $\hat{f}(\mathbf{z})$ at each one.

The surrogate is trained on $\hat{f}$'s **predictions**, not on the true labels y . We are explaining **the model**, not the world.

x_1 x_2 d
0:3-0 | 11
2.5 | 2.2 | $\hat{y}_2$



Sampled points spread across the feature space. LMU IML (CC BY 4.0).

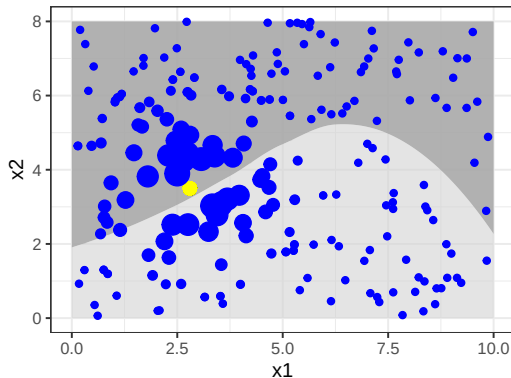
Step 3: weight each sample by how close it is

Those samples are scattered everywhere — but we only care about the neighbourhood of $\mathbf{x}$.

So weight every point by the proximity kernel $\pi(\mathbf{z}) = \exp(-d(\mathbf{x}, \mathbf{z})^2/\sigma^2)$:

- ▶ Close to $\mathbf{x} \Rightarrow$ **big** weight (big dots).
- ▶ Far away $\Rightarrow$ weight ≈ 0 , effectively ignored.

This is the step that makes it “local” — and the kernel width σ is the knob that decides how big the neighbourhood is.



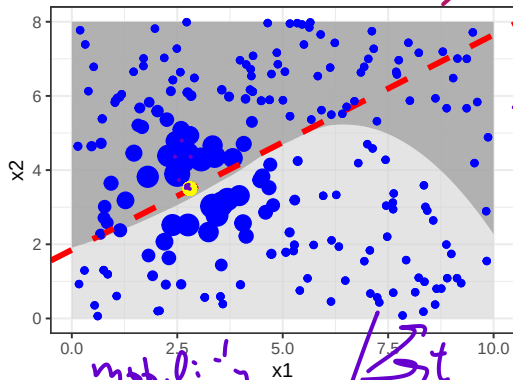
Dot size = proximity weight. LMU IML (CC BY 4.0).

Step 4: fit the interpretable model — and read it

Fit the surrogate $\hat{g}$ on the sampled points, using the black box's predictions as the target and the proximity as **sample weights**.

Here that is a logistic regression: the red line is its local decision boundary. It is wrong far away — and we do not care, because those points had weight ≈ 0 .

The explanation is $\hat{g}$'s coefficients:
“these few features, with these signs, are what moves the prediction *around this point*.”



The fitted local surrogate. LMU IML (CC BY 4.0).

LIME's real trick: explain in a *simplified* space

Those four steps ran in the *raw* feature space — fine for two numeric columns. In general LIME fits the surrogate on an **interpretable representation** instead: a binary vector $\mathbf{z}' \in \{0, 1\}^{d'}$ saying *which pieces of the input are switched on*.

That is what lets the same four steps run on **anything** you can chop into pieces:

	tabular	text	images
one “piece”	a feature (column)	a word / token	a superpixel (patch)
switch it off	resample the column	delete the word	grey the patch out
you read	a weight per feature	highlighted words	highlighted patches

This is why **one** implementation covers all three — and why the explanation always comes back in **human units** (words, patches), never in raw pixels or TF-IDF numbers.

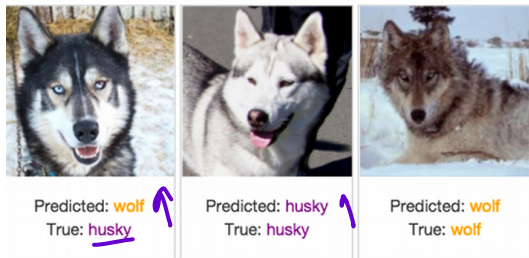
Same move as SHAP's masking slide. Both methods need a notion of “absent”, and both get it by **switching pieces off** — that shared trick is what makes them modality-agnostic.

LIME on images: the husky that was really snow

On images the “features” are **superpixels** (patches), and a sampled point $\mathbf{z}$ is the image with some patches **switched off**. The surrogate then says *which patches* carried the prediction.

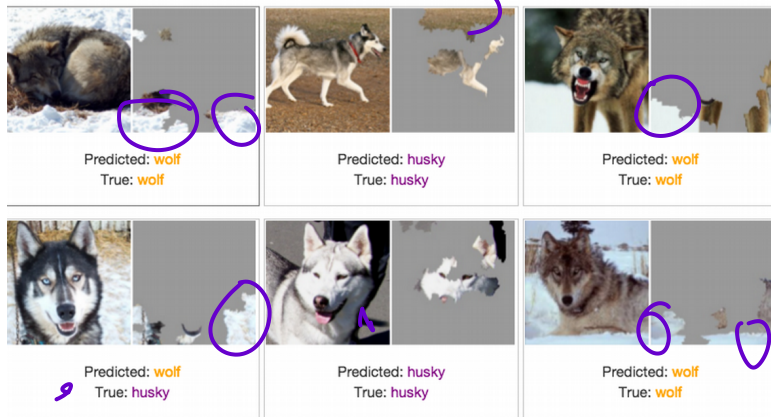
A husky-vs-wolf classifier looked convincing — on the six examples we are about to see, it gets five right. LIME showed what it actually keyed on:

The snow in the background — not the animal. A *clever-Hans* model: right answer, wrong reason, caught only by a local explanation.



Husky vs. wolf: LIME highlights snow pixels. LMU IML (CC BY 4.0).

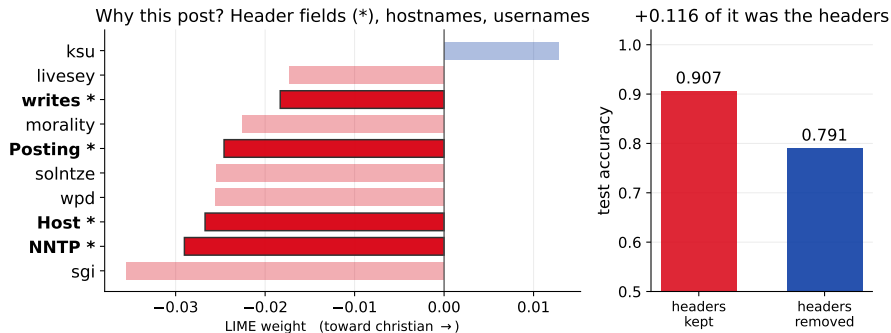
LIME's proof: the explanation *is* the snow



In each pair: the photo (left) and **LIME's explanation** (right — the kept superpixels). The model's "reason" is the **snow**, not the animal, so the snowy **husky** (bottom-left) is misread as a **wolf**. LMU IML (CC BY 4.0) / Ribeiro et al. 2016.

The same trap, in text: a classifier that read the mail headers

Ribeiro et al. ran the *same* experiment on **documents** — *atheism* vs *christianity* newsgroups. Their classifier scored **94%** held-out, top LIME words Posting, Host, Re. Ours agrees: NNTP (a newsgroup header field) is in **21.6%** of training posts (**99.6%** atheism), and solntze/wpd/livesey are one prolific poster's machine and surname (**100%** atheism). Only morality is about the topic.



The snow and the mail header are the same bug — a shortcut that tracks the label *in this dataset*, invisible to held-out accuracy because the artifact is in the test set too. Only a **local explanation** showed it.

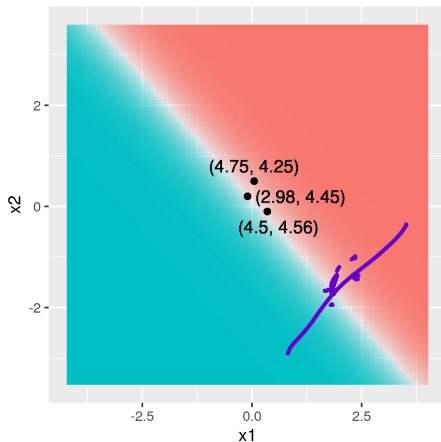
LIME caveats, and SHAP vs. LIME

Caveats — both trace back to step 2 and step 3:

- ▶ **Unstable**: the samples are *random*, so re-running LIME on the same point can give a different explanation.
- ▶ **The neighbourhood is your choice**: the kernel width σ has no “correct” value, and changing it changes the answer. There is no way to check it from the data.

SHAP vs. LIME: SHAP has theory (consistency) + richer plots, but is slower; LIME is simpler/faster but less stable. **SHAP is the modern default.**

$$e^{-\frac{d^2}{2\sigma^2}}$$



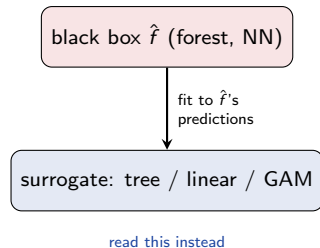
Repeated LIME fits near one point **disagree** — unstable. LMU IML.

LIME's global sibling: the surrogate model

LIME fits a simple model *locally*, around one prediction. Its **global** counterpart does the same for the *whole* model:

Global surrogate: train an interpretable model — a shallow tree, a linear model, a GAM (*generalized additive model* — a sum of one smooth curve per feature, so it bends but stays readable) — to **mimic the black box's predictions** everywhere, then read the surrogate instead of the black box.

- ▶ Check its **fidelity** first (R^2 of surrogate vs. black-box output): a poor mimic is a misleading explanation.
- ▶ In effect it **distills the forest back into deck 1's glass box**.



Counterfactual explanations

Not *why* the model decided — but *what would change* the outcome.

The smallest change that flips the prediction

SHAP and LIME answer “why this prediction?” A **counterfactual** answers a different, often more useful question:

What is the smallest change to the input that would flip the prediction?

- ▶ “Flip” needs a **decision**, so we switch from the (regression) bike model to the **ch. 3 bank-marketing** classifier — *will this client subscribe?* (realistic model, the leaky duration feature removed).
- ▶ A counterfactual is **actionable**: it tells you *what to change*, not just what mattered.

Example: a client predicted **won't subscribe** — the counterfactual says that *had the bank called in a different month*, the model would flip to **will subscribe**.

$0.394 \rightarrow 0.47$

Writing it as an optimization problem

2.1 0.1 → 0

Notation: $\mathbf{x}$ is the **factual** point (the client the model says *won't subscribe*); y' is the **desired** outcome ("subscribes"), possibly an interval. We search for a counterfactual $\mathbf{x}'$.

A **valid** counterfactual must do two things at once, so we penalise both:

X, 0.1
7

$$\arg \min_{\mathbf{x}'} \lambda_1 \underbrace{o_{\text{target}}(\hat{f}(\mathbf{x}'), y')}_{\text{does it flip?}} + \lambda_2 \underbrace{o_{\text{prox}}(\mathbf{x}', \mathbf{x})}_{\text{is it a small change?}}$$

0.1
4.4.

Distance in target space o_{target} :

- ▶ Regression / predicted probability:
 $|\hat{f}(\mathbf{x}') - y'|$
- ▶ Hard labels: $\mathbb{1}_{\{\hat{f}(\mathbf{x}') \neq y'\}}$

λ_1, λ_2 trade the two off. Push λ_1 up and you always flip the prediction — possibly by changing everything.

X → (x', y')

0	1
1	0

Distance in *feature* space is the harder one, because features have different types and units — next slide.

Distance in feature space: the Gower distance

A bank row mixes numbers (balance, age) with categories (job, month). **Gower** handles both, and normalises each to $[0, 1]$:

1 0 0 - 1

$$o_{\text{prox}}(\mathbf{x}', \mathbf{x}) = \underline{\text{Gower}}(\mathbf{x}', \mathbf{x}) = \frac{1}{p} \sum_{j=1}^p \delta_G(x'_j, x_j) \in [0, 1]$$

0 x 1

$$\delta_G(x'_j, x_j) = \begin{cases} \frac{|x'_j - x_j|}{\hat{R}_j} & x_j \text{ numerical} \\ \mathbb{1}_{\{x'_j \neq x_j\}} & x_j \text{ categorical} \end{cases}$$

$\hat{R}_j$ is feature j 's range in the training data. Dividing by it is what makes “+500 in balance” and “+3 years of age” comparable at all.

A categorical feature contributes 0 or 1 — nothing in between. Changing month from June to April costs the same as changing it to December.

Text and images have no Gower distance — a sentence has no ranges and no categories. There “a small change” becomes the smallest **edit** (swap one word), and *plausibility* becomes “is it still a grammatical sentence?” — much harder to optimise than a number.

Two more objectives: sparsity and plausibility

Sparsity — change *few* features. People act on short advice, not on “adjust these nine things”. Count the changes (L_0):


$$o_{\text{sparse}}(\mathbf{x}', \mathbf{x}) = \sum_{j=1}^p \mathbb{1}_{\{x'_j \neq x_j\}}$$

L_0

Plausibility — the suggestion must be a *realistic* row, not an impossible one (deck 2's extrapolation trap again). Proxy: stay close to real data, via the nearest training point $\mathbf{x}^{[1]}$:

$$o_{\text{plaus}}(\mathbf{x}', \mathbf{X}) = \text{Gower}(\mathbf{x}', \mathbf{x}^{[1]})$$

Putting all four together gives the objective the libraries actually optimise:

$$\arg \min_{\mathbf{x}'} \lambda_1 o_{\text{target}}(\hat{f}(\mathbf{x}'), y') + \lambda_2 o_{\text{prox}}(\mathbf{x}', \mathbf{x}) + \lambda_3 o_{\text{sparse}}(\mathbf{x}', \mathbf{x}) + \lambda_4 o_{\text{plaus}}(\mathbf{x}', \mathbf{X})$$

Actionability is the fifth wish, and it is *not* a λ — it is a **constraint**: freeze the features the person cannot change (age, race), and optimise only over the rest.

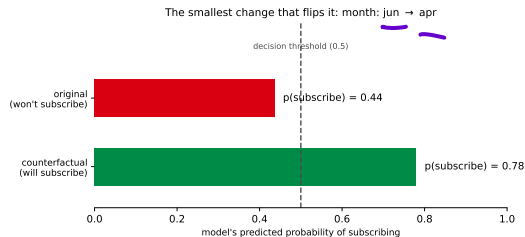
The five wishes, and one real counterfactual



Each objective term is one of the things we want:

- Validity** O_{target} — it actually flips
- Proximity** O_{prox} — a small change
- Sparsity** O_{sparse} — few features
- Plausibility** O_{plaus} — a realistic row
- Actionability** a *constraint*, not a term

They pull against each other: the closest counterfactual is often the least plausible one.



Real DiCE counterfactual (bank): age, job, balance, outcome **frozen** (actionability); only the call month moved — one change, and the model crosses the decision line.



Two warnings before you hand one to a customer

1. The Rashomon effect

- ▶ The optimum is **not unique** — many equally close, equally valid counterfactuals exist.
- ▶ So many equally good “explanations” exist for the same decision.
- ▶ A nonlinear model can even offer **contradictory** advice (raise *or* lower the same feature). Which one do you show?

Still an open problem in IML.

2. It explains the model, not the world

- ▶ “Wait 5 years and you’d be approved” — but in 5 years the applicant’s *job*, *income* and *balance* have changed too. Features are **causally linked**; the counterfactual moved one in isolation.
- ▶ **Model drift**: the bank retrain, and the advice you gave is now invalid.

A counterfactual is a statement about a **fitted function**, not a promise about reality.

Counterfactuals in code

```
import dice_ml
d    = dice_ml.Data(dataframe=df, continuous_features=num,
                    outcome_name="y")
m    = dice_ml.Model(model=clf, backend="sklearn")    # any fitted
                    classifier
exp  = dice_ml.Dice(d, m, method="random")

cf = exp.generate_counterfactuals(
    client, total_CFs=3, desired_class="opposite",
    features_to_vary=["contact", "month", "campaign"]) #
                    actionable only
cf.visualize_as_dataframe()
```

Libraries: **DiCE** (diverse counterfactuals), **Alibi**. Constrain `features_to_vary` so the advice stays *actionable*.

Same data, four rankings

One bike dataset, (mostly) one model — four “importance” methods, four different top-3s:

Method	Top 3 features
Linear coefficients (deck 1)	yr, atemp, season
Tree impurity (deck 1)	temp, yr, atemp
Permutation / PFI (deck 2)	yr, atemp, temp
SHAP (deck 3)	yr, temp, hum

They mostly agree on yr/temperature, but disagree on the rest (season? hum? temp vs atemp?). **That** is what “importance is method-dependent” means — always say *which* importance, on *what* data.

Which method when? (the whole chapter)

You want. . .	Reach for. . .
to read the model itself	a glass-box model — coefficients / a tree / rules (deck 1)
global “which features matter”	PFI or SHAP bar (decks 2–3)
“how does a feature act”	PDP / ICE or SHAP dependence (decks 2–3)
“why this one prediction”	SHAP waterfall or LIME (deck 3)
“what would change the outcome”	counterfactuals — DiCE / Alibi (deck 3)

Whatever you pick: **(1)** correlated features mislead them all, **(2)** different methods rank differently (previous slide), **(3)** none of it is *causation*. **(4)** None of them is tabular-only — on text the “features” are **words**, on images **superpixels**; everything above still applies.

Recap

- ▶ **SHAP** borrows the **Shapley value** from **cooperative game theory** (Shapley, 1953): features are players, the payout is $\hat{f}(\mathbf{x}) - \mathbb{E}[\hat{f}]$, and each feature gets its average marginal contribution. So $\hat{f}(\mathbf{x}) = \text{base} + \sum_j \phi_j$.
- ▶ Read the four plots: **waterfall** (one prediction), **bar** (global importance), **beeswarm** (global, colour = value), **dependence** (effect shape + interaction).
- ▶ **TreeSHAP** makes it the default for trees/boosting; SHAP **unifies** PFI & PDP and adds a local view.
- ▶ **LIME** fits a local surrogate: **sample** around the point, **weight** by proximity $\pi(\mathbf{z})$, **fit** a sparse linear model on the black box's own predictions. Handy, but the randomness and the kernel width σ make it **unstable**.
- ▶ **Any modality**: both only need a way to switch one piece of the input **off** — a column, a **word**, a **superpixel**. One tool, snow-detector *and* mail-header detector.
- ▶ **Counterfactuals** answer “what would change the outcome”, as an optimization trading **validity** against **proximity**, **sparsity** and **plausibility** — actionability being a **constraint**, not a term. Beware the **Rashomon effect**: the answer is not unique.
- ▶ Everything explains the **model**, not causation — and plots are easy to over-read.

That's interpretability: read the model when you can, explain it when you can't — importance, effects, and per-prediction attributions, each answering a different question. **Next**: the chapter practical — *explaining startup success*.